

# Euclidean Algorithms

## 1 Goal

### Problem

Given two integers  $a$  and  $b$ , compute their greatest common divisor:

$$\gcd(a, b).$$

The extended version also finds integers  $x$  and  $y$  such that

$$ax + by = \gcd(a, b).$$

The ordinary Euclidean Algorithm gives the value of the gcd. The Extended Euclidean Algorithm gives the gcd and also a linear representation of it.

## 2 Euclidean Algorithm

### Main Identity

For integers  $a$  and  $b > 0$ ,

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Write the division equation as

$$a = bq + r, \quad 0 \leq r < b.$$

Here  $q = \lfloor a/b \rfloor$  and  $r = a \bmod b$ .

The reason the identity works is that every common divisor of  $a$  and  $b$  also divides

$$r = a - bq,$$

and every common divisor of  $b$  and  $r$  also divides

$$a = bq + r.$$

So the set of common divisors does not change.

## 3 Algorithm

### Euclidean Algorithm

```
Euclid(a, b):  
    if b == 0:  
        return a  
    return Euclid(b, a mod b)
```

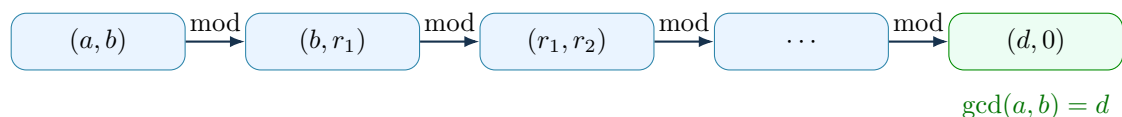
The algorithm repeatedly replaces the larger pair

$$(a, b)$$

by the smaller pair

$$(b, a \bmod b).$$

It stops when the second value becomes 0. The first value at that moment is the gcd.



#### Small Recursive Trace

$$\text{Euclid}(30, 21) = \text{Euclid}(21, 9) = \text{Euclid}(9, 3) = \text{Euclid}(3, 0) = 3.$$

This call sequence makes three recursive calls.

## 4 Worked Example: $\text{gcd}(99, 78)$

Use repeated division:

$$99 = 78(1) + 21,$$

$$78 = 21(3) + 15,$$

$$21 = 15(1) + 6,$$

$$15 = 6(2) + 3,$$

$$6 = 3(2) + 0.$$

The last nonzero remainder is 3, so

$$\boxed{\text{gcd}(99, 78) = 3.}$$

| Step | Dividend | Divisor | Remainder |
|------|----------|---------|-----------|
| 1    | 99       | 78      | 21        |
| 2    | 78       | 21      | 15        |
| 3    | 21       | 15      | 6         |
| 4    | 15       | 6       | 3         |
| 5    | 6        | 3       | 0         |

## 5 Extended Euclidean Algorithm

### Bezout's Identity

For integers  $a$  and  $b$ , there exist integers  $x$  and  $y$  such that

$$\boxed{ax + by = \text{gcd}(a, b).}$$

The integers  $x$  and  $y$  are called **Bezout coefficients**. They are not always unique, but one valid pair is enough for modular inverses and many cryptographic algorithms.

## 6 Coefficient Recurrence

Suppose the recursive call returns

$$d = \gcd(b, a \bmod b)$$

with coefficients  $x_1, y_1$ , so

$$bx_1 + (a \bmod b)y_1 = d.$$

Since

$$a \bmod b = a - \left\lfloor \frac{a}{b} \right\rfloor b,$$

substitute:

$$bx_1 + \left( a - \left\lfloor \frac{a}{b} \right\rfloor b \right) y_1 = d.$$

Rearrange into the form  $ax + by = d$ :

$$ay_1 + b \left( x_1 - \left\lfloor \frac{a}{b} \right\rfloor y_1 \right) = d.$$

Therefore,

$$x = y_1, \quad y = x_1 - \left\lfloor \frac{a}{b} \right\rfloor y_1.$$

### Extended Euclidean Algorithm

```
ExtendedEuclid(a, b):  
    if b == 0:  
        return (a, 1, 0)  
  
    (d, x1, y1) = ExtendedEuclid(b, a mod b)  
  
    x = y1  
    y = x1 - floor(a / b) * y1  
  
    return (d, x, y)
```

The returned triple  $(d, x, y)$  satisfies

$$d = \gcd(a, b), \quad ax + by = d.$$

## 7 Back-Substitution Example

### Problem

Find integers  $x$  and  $y$  such that

$$99x + 78y = \gcd(99, 78).$$

### Solution

From the Euclidean Algorithm:

$$99 = 78(1) + 21, \quad 78 = 21(3) + 15,$$

$$21 = 15(1) + 6, \quad 15 = 6(2) + 3.$$

Start with the last nonzero remainder:

$$3 = 15 - 2(6).$$

Now replace each remainder using the previous division equation:

$$6 = 21 - 15.$$

So

$$3 = 15 - 2(21 - 15) = 3(15) - 2(21).$$

Next,

$$15 = 78 - 3(21),$$

hence

$$3 = 3(78 - 3(21)) - 2(21) = 3(78) - 11(21).$$

Finally,

$$21 = 99 - 78,$$

so

$$3 = 3(78) - 11(99 - 78).$$

Therefore,

$$3 = -11(99) + 14(78).$$

### Answer

$$99(-11) + 78(14) = 3$$

Thus one valid Extended Euclidean output is

$$(d, x, y) = (3, -11, 14).$$

## 8 Back-Substitution Flow

$$3 = 15 - 2(6)$$

↓replace 6

$$3 = 3(15) - 2(21)$$

↓replace 15

$$3 = 3(78) - 11(21)$$

↓replace 21

$$3 = -11(99) + 14(78)$$

## 9 Extended Euclid Trace Table

### Trace for ExtendedEuclid(99, 78)

Back substitution is easier by hand. This table shows the same computation in the recursive form used by the algorithm.

Each row records one recursive level. The returned triple  $(d, x, y)$  satisfies

$$d = ax + by.$$

| $a$ | $b$ | $\lfloor a/b \rfloor$ | $d$ | $x$ | $y$ |
|-----|-----|-----------------------|-----|-----|-----|
| 99  | 78  | 1                     | 3   | -11 | 14  |
| 78  | 21  | 3                     | 3   | 3   | -11 |
| 21  | 15  | 1                     | 3   | -2  | 3   |
| 15  | 6   | 2                     | 3   | 1   | -2  |
| 6   | 3   | 2                     | 3   | 0   | 1   |
| 3   | 0   | -                     | 3   | 1   | 0   |

### Returned result

The first row gives

$$(d, x, y) = (3, -11, 14),$$

so

$$99(-11) + 78(14) = 3.$$

## 10 Why the Algorithms Stop

In each Euclidean step,

$$a = bq + r, \quad 0 \leq r < b.$$

The second argument strictly decreases:

$$b > r \geq 0.$$

Since nonnegative integers cannot decrease forever, the process must eventually reach remainder 0.

## 11 Running Time

Each recursive call replaces

$$(a, b) \text{ by } (b, a \bmod b).$$

If  $a < b$ , the first call swaps the order:

$$\text{Euclid}(a, b) \rightarrow \text{Euclid}(b, a).$$

So we may analyze the case

$$a > b \geq 0.$$

### Fibonacci Worst-Case Lemma

If  $a > b \geq 1$  and  $\text{Euclid}(a, b)$  performs  $k \geq 1$  recursive calls, then

$$a \geq F_{k+2} \quad \text{and} \quad b \geq F_{k+1}.$$

This lemma says: many recursive calls require large inputs. The slowest cases happen when the remainders decrease like Fibonacci numbers.

### Lamé's Theorem

For any integer  $k \geq 1$ , if

$$a > b \geq 1 \quad \text{and} \quad b < F_{k+1},$$

then  $\text{Euclid}(a, b)$  makes fewer than  $k$  recursive calls.

The upper bound is tight. The call

$$\text{Euclid}(F_{k+1}, F_k)$$

makes exactly  $k - 1$  recursive calls, because

$$F_{k+1} \bmod F_k = F_{k-1}.$$

### Arithmetic-operation count

Since  $F_k$  grows exponentially with  $k$ , the number of recursive calls is logarithmic:

$$\text{number of Euclid calls} = \mathcal{O}(\log b).$$

### Bit-operation cost

If  $a$  and  $b$  are  $\beta$ -bit integers, then  $\text{Euclid}$  performs  $\mathcal{O}(\beta)$  arithmetic operations. With schoolbook multiplication/division costing  $\mathcal{O}(\beta^2)$  bit operations, this gives

$$\mathcal{O}(\beta^3) \text{ bit operations.}$$

A tighter analysis can improve the bit bound, but this is the basic estimate from the textbook discussion.

## 12 Review Box

### Core Facts

$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ .  
Stop when the second argument becomes 0.  
Last nonzero remainder is the gcd.  
 $ax + by = \text{gcd}(a, b)$  for some integers  $x, y$ .  
 $x = y_1, \quad y = x_1 - \lfloor a/b \rfloor y_1$ .  
 $\text{Euclid}(a, b)$  makes  $\mathcal{O}(\log b)$  recursive calls.